

UNIVERSIDAD SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE
DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
ESTRUCTURA DE DATOS



Mauricio Joel Gómez Barrios - 202031478

24 de febrero de 2026

Análisis de Complejidad.

Este es el análisis de rendimiento de los dos algoritmos principales.

1) Algoritmo de Barajado del Mazo (Pila<T>::barajar):

El método extrae todos los datos de la pila hacia un arreglo dinámico temporal, luego se posiciona en la última posición y elige la posición de una carta al azar en el arreglo, incluida así misma, luego cambia de lugar la última carta con la que salió en el número al azar (la carta puede intercambiarse consigo misma) luego pasa a la penúltima carta e iterando desde el último elemento hasta el primero para intercambiar las cartas en posiciones aleatorias, por último vuelve a recorrer la pila para reasignar los datos ya mezclados a los nodos.

Analizando la mezcla se hace con un bucle for ($O(n)$) y la reasignación a la pila original ($O(n)$) así que podemos decir que la complejidad es de **Tiempo Lineal $O(n)$** .

Bastante eficiente para mezclar mazos ya que el tiempo de ejecución crece de forma proporcional a la cantidad de cartas haciendo una mezcla instantánea ya sea con 1 o 10 mazos.

```
void barajar()
{
    if (tamaño < 2) return;
    T* arr = new T[tamaño];
    Nodo<T>* actual = tope;
    int count = 0;
    while(actual != nullptr)
    {
        arr[count++] = actual->dato;
        actual = actual->siguiente;
    }
    for (int i = tamaño - 1; i > 0; i--)
    {
        int j = rand() % (i + 1);
        T temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
    actual = tope;
    count = 0;
    while(actual != nullptr)
    {
        actual->dato = arr[count++];
        actual = actual->siguiente;
    }
    delete[] arr;
}
```

2) Búsqueda de cartas en la Mano (Jugador::tieneCarta y aplicarDestruccionExacta):

Para buscar una carta específica en la mano del jugador, el sistema utiliza un bucle (ya sea for o while) que recorre la cantidad total de cartas. Dentro de ese bucle, se llama a la función `mano->obtenerPorIndice(i)`.

El método obtenerPorIndice(i) debe recorrer los nodos desde la cabeza hasta el índice. Donde el bucle principal recorre la cantidad total de las cartas y la búsqueda interna recorre i veces en cada iteración, eso resulta en una sumatoria un poco mas compleja dandonos una complejidad de **Tiempo Cuadrático $O(k^2)$** , lo que parece ser muy lento pero al manejar los mazos de los jugadores se esperarían cantidades de cartas más manejables y no tan grandes.

tieneCarta:

```
bool Jugador::tieneCarta(TipoCarta tipo)
{
    for(int i = 0; i < mano->getTamaño(); i++)
    {
        Carta* c = mano->obtenerPorIndice(i);
        if(c->getTipo() == tipo)
        {
            return true;
        }
    }
    return false;
}
```

aplicarDestruccionExacta:

```
int Jugador::aplicarDestruccionExacta(Color col, TipoCarta tip, int val)
{
    int i = 0;
    int destruidas = 0;
    while (i < mano->getTamaño())
    {
        Carta* carta = mano->obtenerPorIndice(i);
        if (carta->getColor() == col && carta->getTipo() == tip && carta->getValor() == val)
        {
            mano->eliminar(carta);
            if (carta->getLadoOscuro() != nullptr)
            {
                delete carta->getLadoOscuro();
            }
            delete carta;
            destruidas++;
        }
        else
        {
            i++;
        }
    }
    return destruidas;
}
```